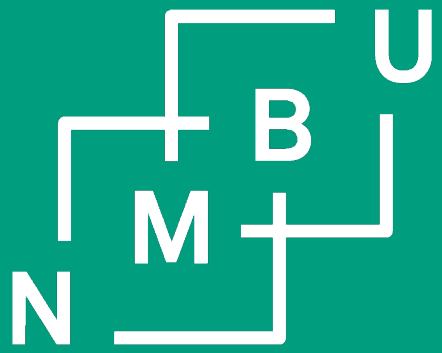
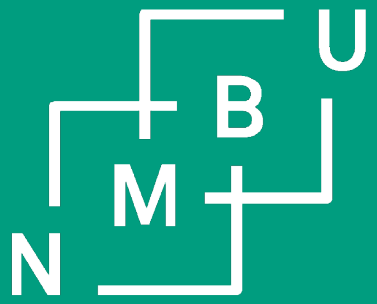

U
B
M
N



Norwegian University
of Life Sciences

Lecture Agenda

- Data inspection/Data exploration/Exploratory Data Analysis (EDA)
  - Visualization with `matplotlib` and `seaborn`
- Two basic ML classification algorithms
  - Perceptron
  - Adaline
- Learning as an optimization problem
  - Gradient Descent
  - Feature scaling to improve Gradient Descent

Raw Data Inspection

DAT200 - Applied Machine Learning

Department of Data Science, Faculty of Science of Technology

Exploratory Data Analysis (EDA)

- An **important first step** in any ML process
- To select appropriate ML methods to use and obtain the best possible results it is important that we **understand the data**
- One of the best ways to understand distributions within the dataset, and how different variables relate to each other is by **visualizing** them
- It is **absolutely necessary** to do this before training your machine learning models

Exploratory Data Analysis in the CAs

- In CA2-5 you will be given a dataset, and tasked with (among other things):
  - Exploring the dataset
  - Preprocessing the data
  - Selecting an appropriate machine learning model
  - Evaluating the models
- EDA is an important step of the CAs, and if you omit it they will **not** be approved

- Outlier removal is an important step of preprocessing -> discovered in EDA
- NOTE: Sometimes, we will add synthetic outliers to the dataset that will negatively affect the performance of ML models

- Let´s look at some examples using the **diabetes dataset** included in scikit-learn

Exploratory Data Analysis: The data

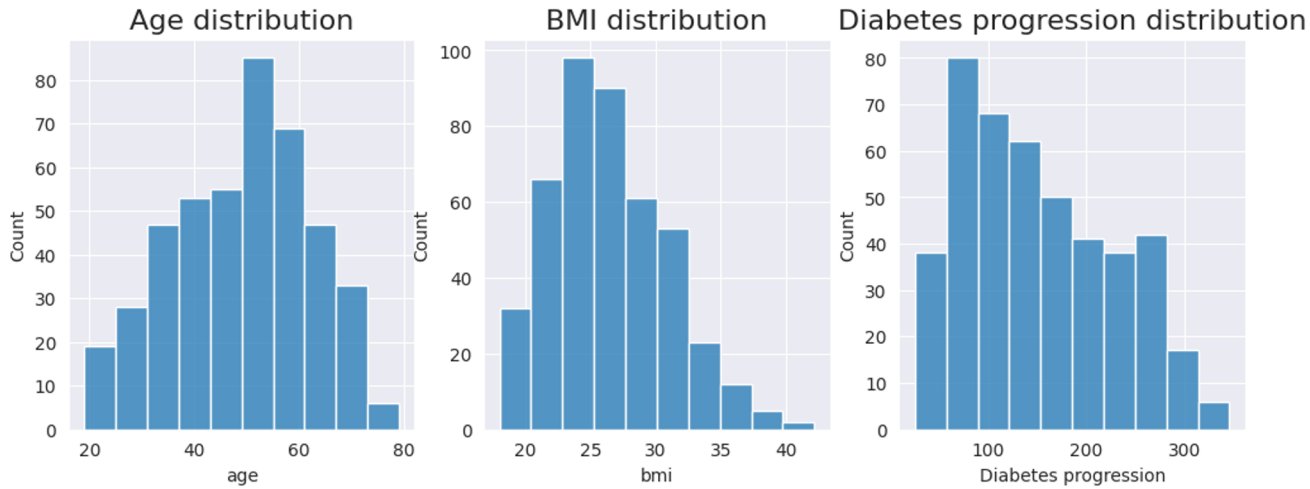
- The diabetes dataset has 10 input columns and one target column
- Input:
  - Age
  - Sex
  - BMI
  - Blood pressure
  - Four different measures of blood cholesterol, together with triglycerides and blood glucose

	age	sex	bmi	bp	tc	ldl	hdl	tch	ltg	glu	Diabetes progression
0	59	male	32.1	101	157	93.2	38	4.00	4.8598	87	151
1	48	female	21.6	87	183	103.2	70	3.00	3.8918	69	75
2	72	male	30.5	93	156	93.6	41	4.00	4.6728	85	141
3	24	female	25.3	84	198	131.4	40	5.00	4.8903	89	206

- Compute **descriptive statistics** to get a first overview of the data

Exploratory Data Analysis: Univariate plots

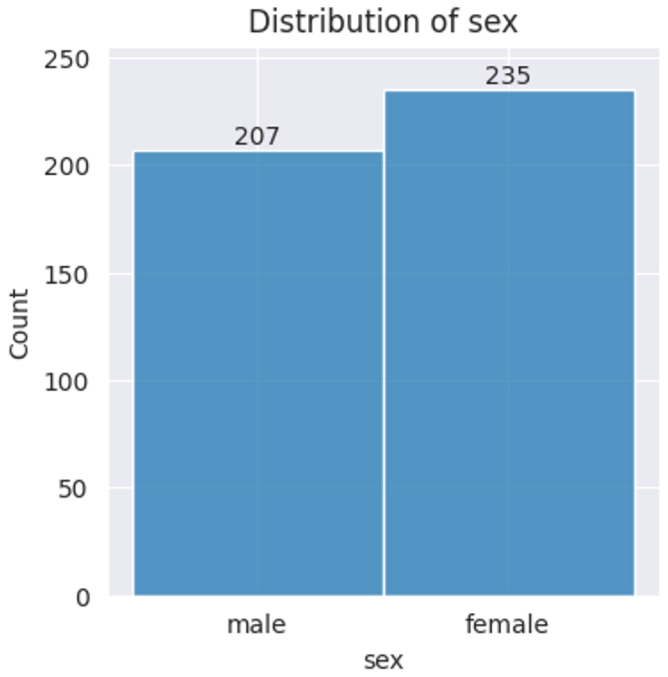
- Histograms
  - Inspect the distribution of each attribute
  - Group data into bins
  - Count the number of observations in each bin
  - Setting an appropriate bin size is important for the histograms to be informative



seaborn.histplot

Exploratory Data Analysis: Univariate plots

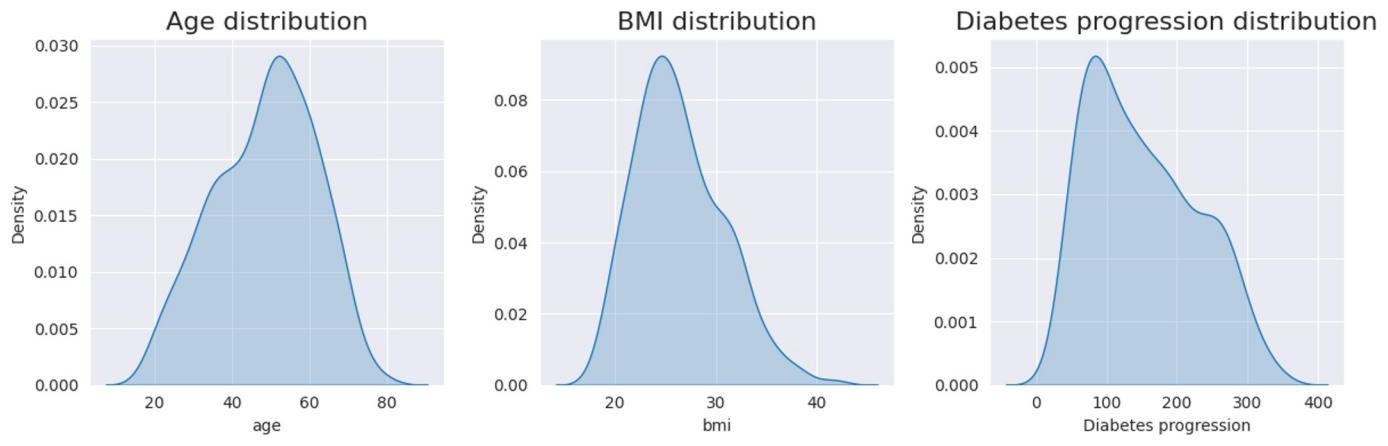
- Can also be used for categorical data
- Can be helpful to add numbers on the bars



seaborn.histplot

Exploratory Data Analysis: Univariate plots

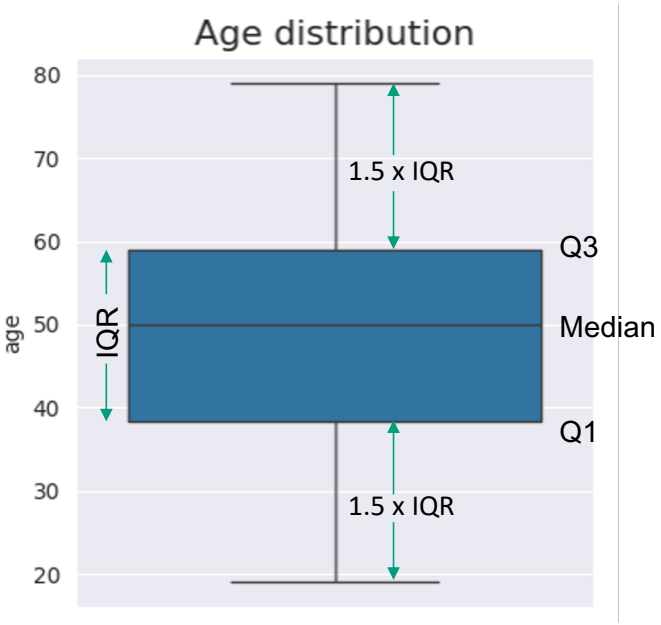
- Density plots
  - Another way of inspecting the distribution
  - Use a smooth continuous curve



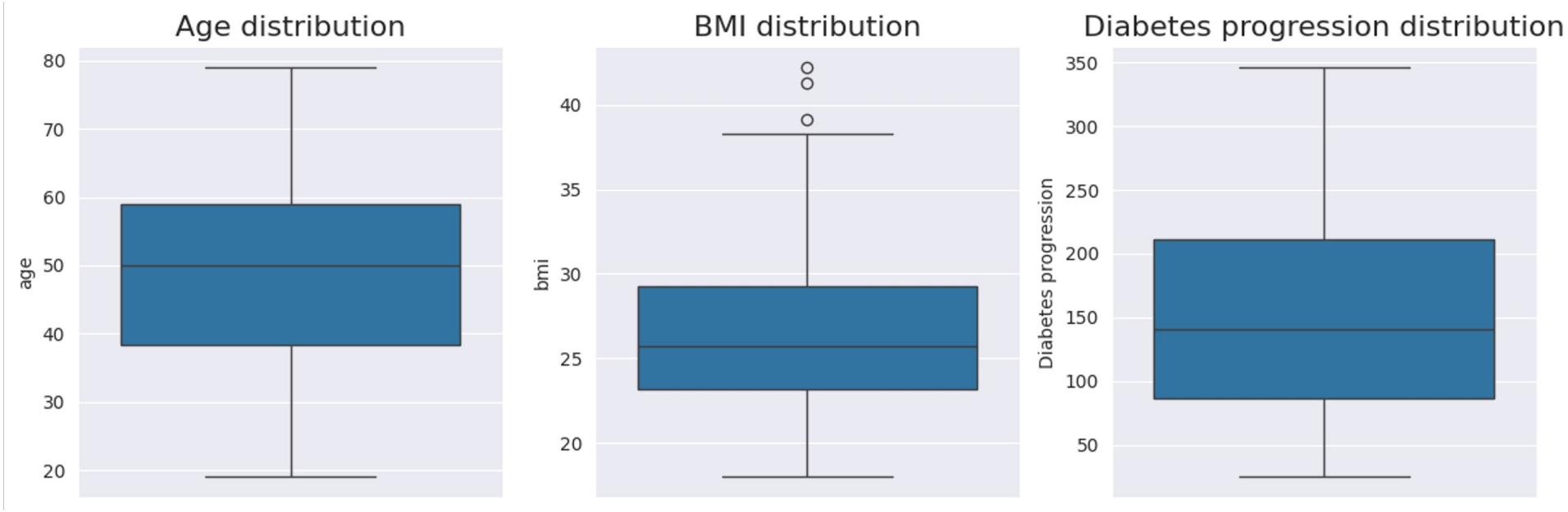
seaborn.kdeplot

Exploratory Data Analysis: Univariate plots

- Box and whisker plots
  - Another way of inspecting the univariate distribution
  - Instead of showing a complete distribution they give a short summary
    - Use a line to represent the median sample/entry
    - The box encompasses the middle 50% of the data, from the 25th to the 75th percentile (Q3-Q1)
    - The “whiskers” are 1.5 times greater than the size of the spread of the middle 50% of the data (1.5 x IQR)
    - Dots outside the “whiskers” represent potential outliers



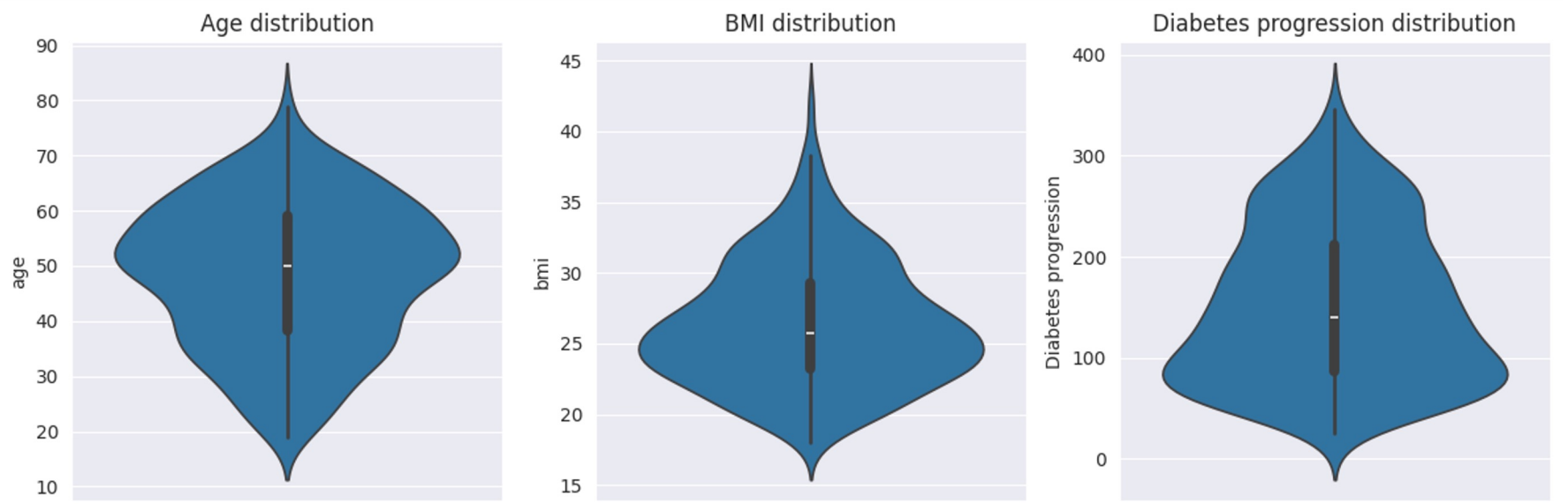
Exploratory Data Analysis: Univariate plots



seaborn.boxplot

Exploratory Data Analysis: Univariate plots

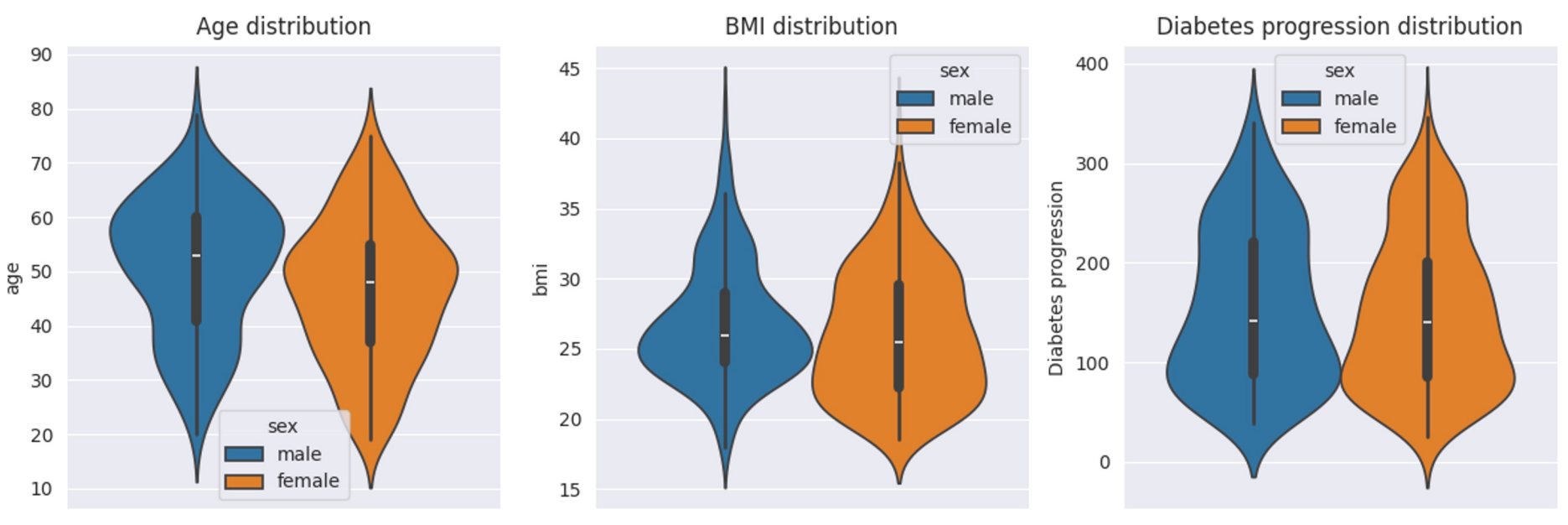
- Violin plots: A better alternative to Box and whisker plots
  - Give a more complete/precise description of the data



seaborn.violinplot

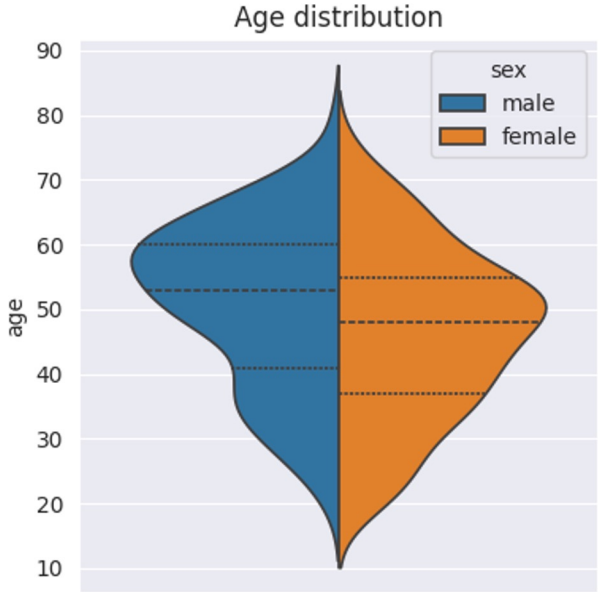
Exploratory Data Analysis: Univariate plots

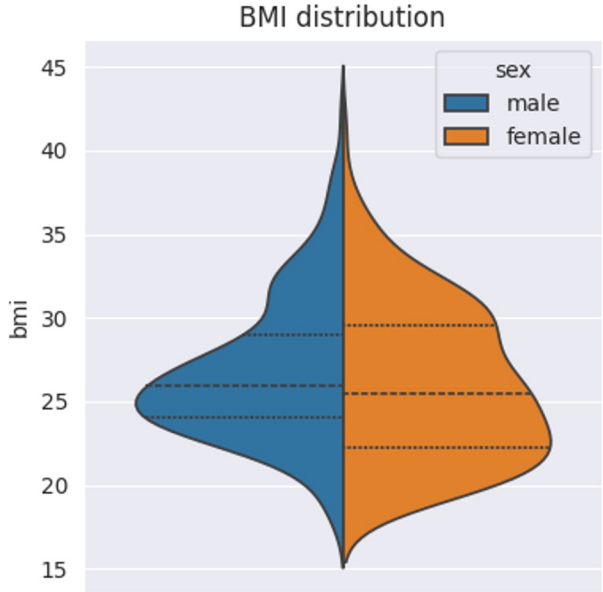
- `Seaborn` allows for *splitting* using the parameter `hue`.
- Note that the *violins* are symmetrical, so the information can be compressed further



Exploratory Data Analysis: Univariate plots

- `Seaborn` allows for *splitting* using the parameter `hue`.
- Here the inner box-visuals are replaced with lines representing the median, 25th and 75th percentiles

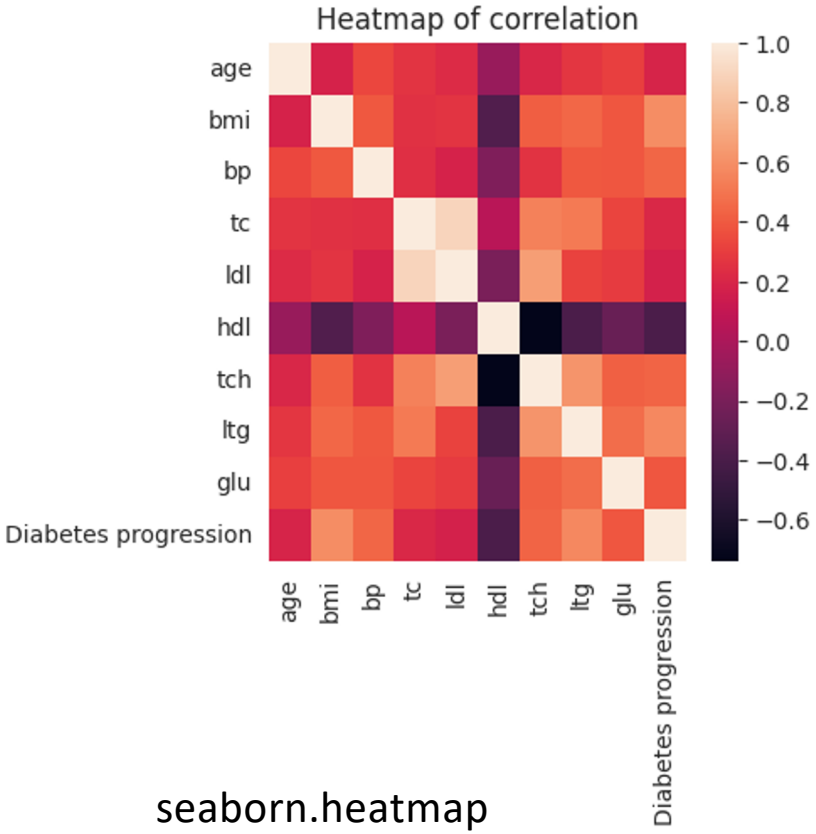






Exploratory Data Analysis: Multivariate plots

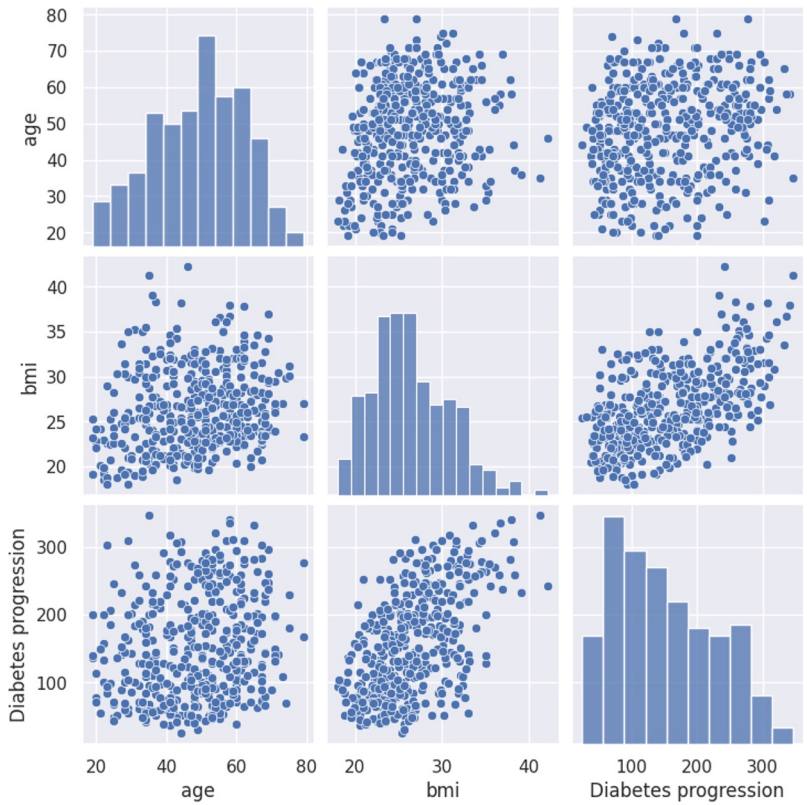
- Heatmap of correlations
  - Indicates how related two variables are
  - **Positive** correlation: Two variables change in **the same** direction
  - **Negative** correlation: Two variables change in **opposite** directions



seaborn.heatmap

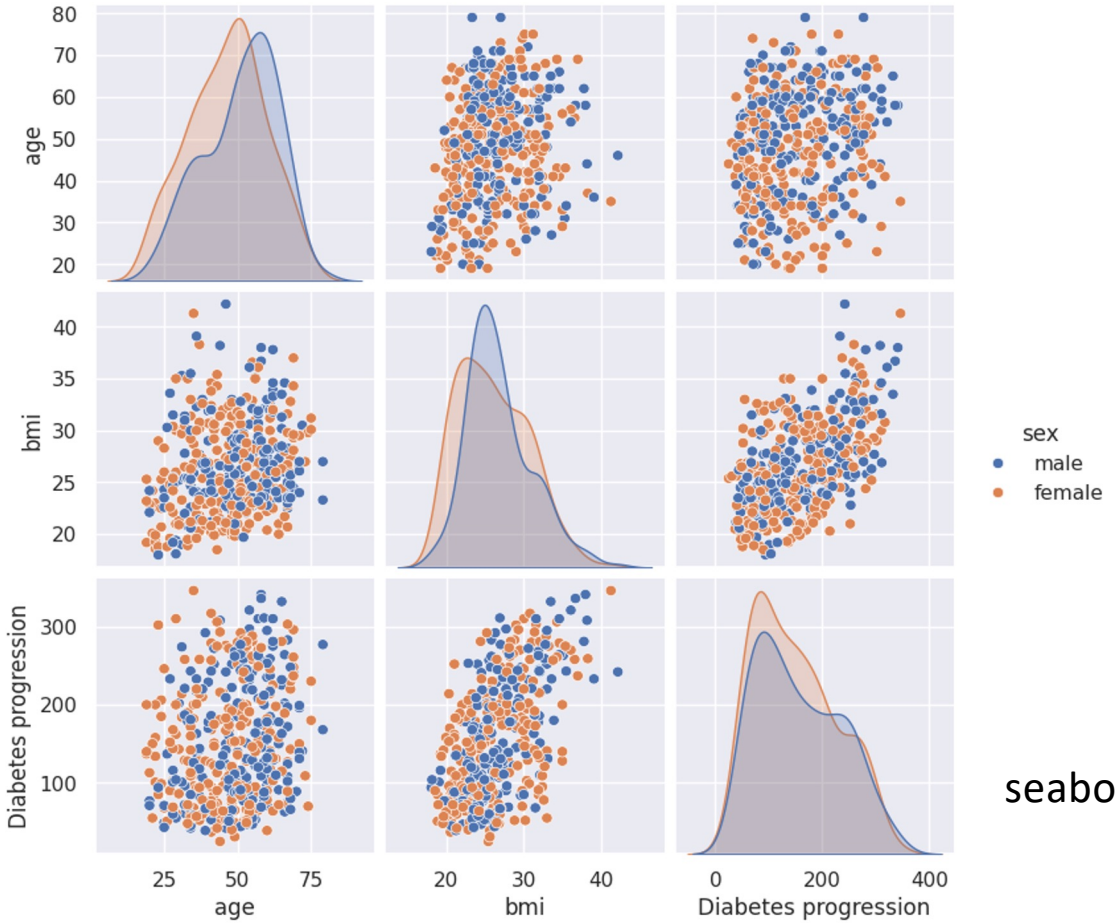
Exploratory Data Analysis: Multivariate plots

- Scatter Plot Matrix/Pair plot
  - Shows the relationship between two variables as dots in two dimensions
  - Useful for spotting structured relationships between variables
  - Structural relationships may also be correlated -> good candidates for removal from the dataset
  - The individual distributions of the variables are shown on the diagonal



seaborn.pairplot

Exploratory Data Analysis: Multivariate plots



seaborn.pairplot

Exploratory Data Analysis: Multivariate plots

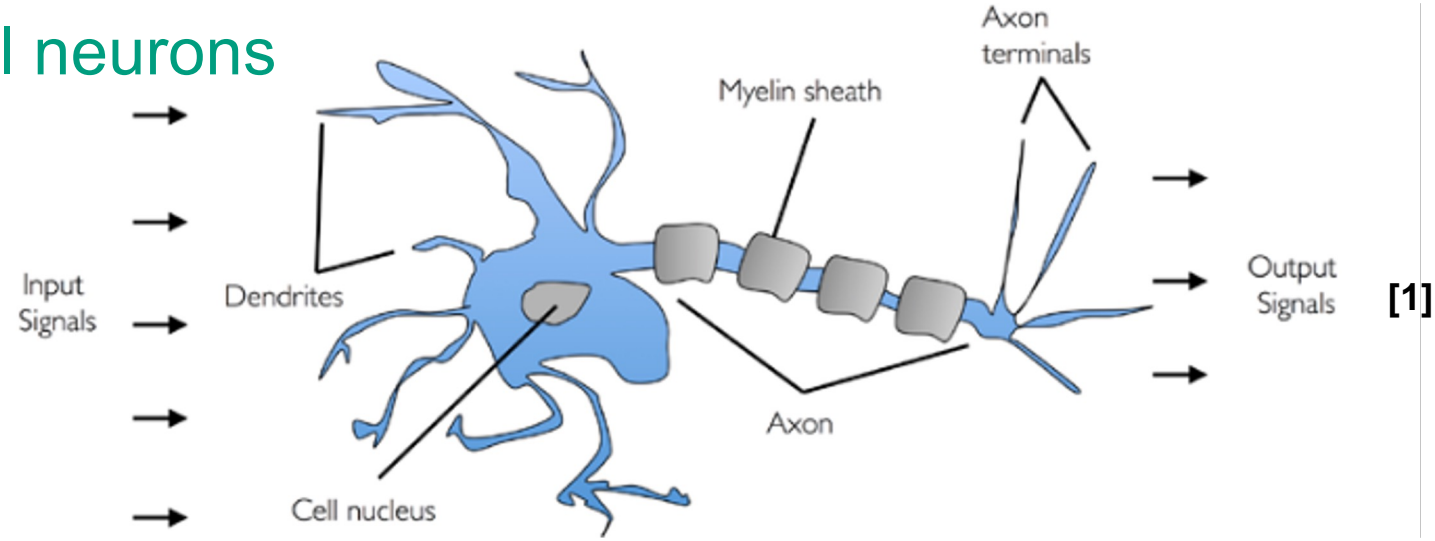
- To see more about why data visualization is important, and summary statistics alone are not enough, check out this article
- The code used to generate the plots can be found in the file:
  - `01_raw_data_inspection.ipynb` under the folder for this lecture on Canvas
  - Check it out after the lecture

Learning as a concept (Ch. 2 in Raschka)

DAT200 - Applied Machine Learning

Department of Data Science, Faculty of Science of Technology

Artificial neurons

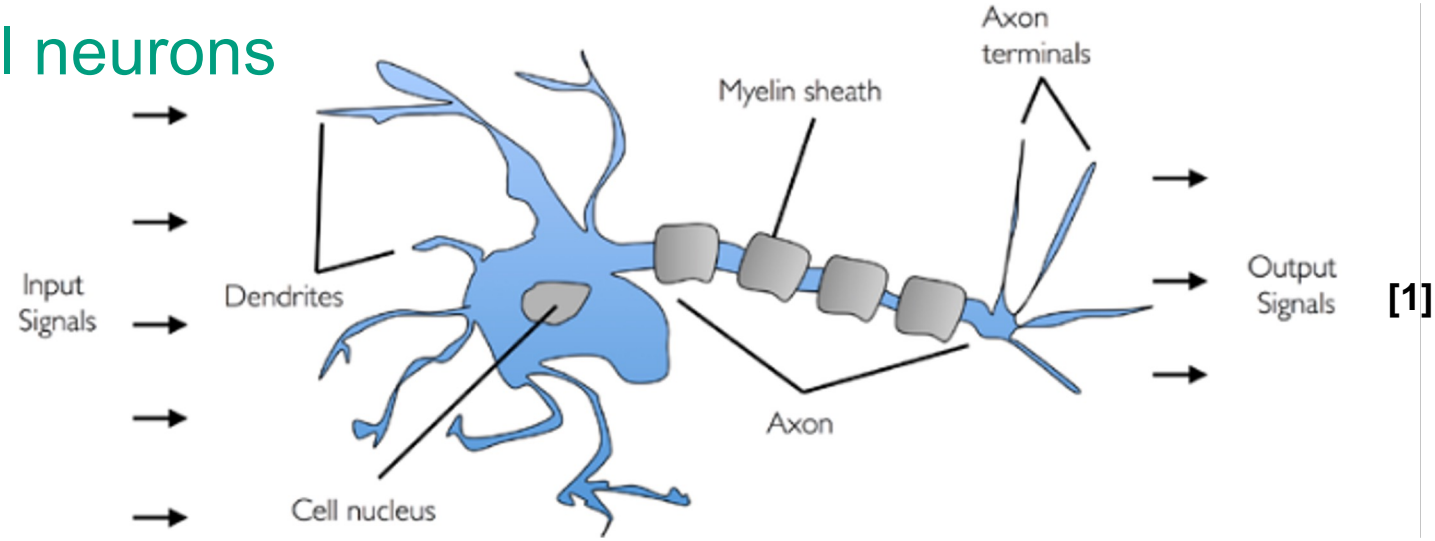


[1]

- The first ML models where inspired by the neurons in the human brain
- Neurons are interconnected nerve cells in the brain
- Neurons are involved in processing and transmitting:
  - Chemical signals
  - Electrical signals
- A neuron acts as a simple logic gate with a binary output

[1] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 20

Artificial neurons

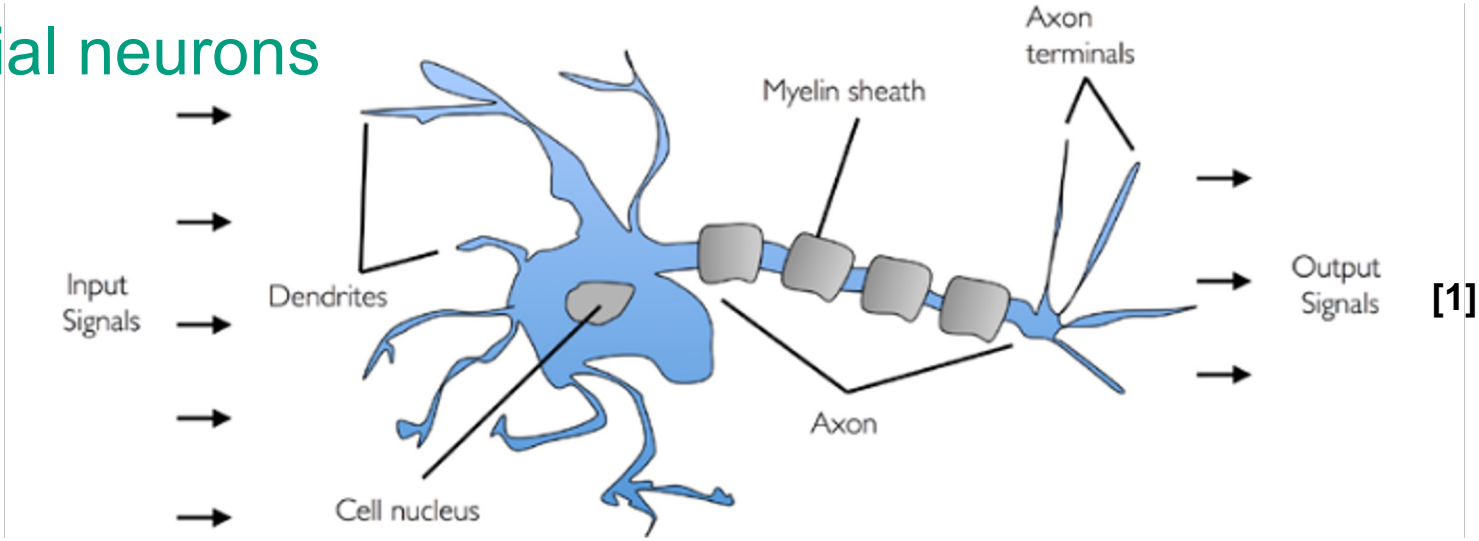


[1]

- Multiple signals arrive at the dendrites
- Signals are integrated into the cell body
- If the accumulated signal exceeds a specific threshold → An output signal is generated and passed on to the axon
- McCullock-Pitts (MCP) neuron: First simplified concept of a brain cell (1943)

[1] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 20

Artificial neurons

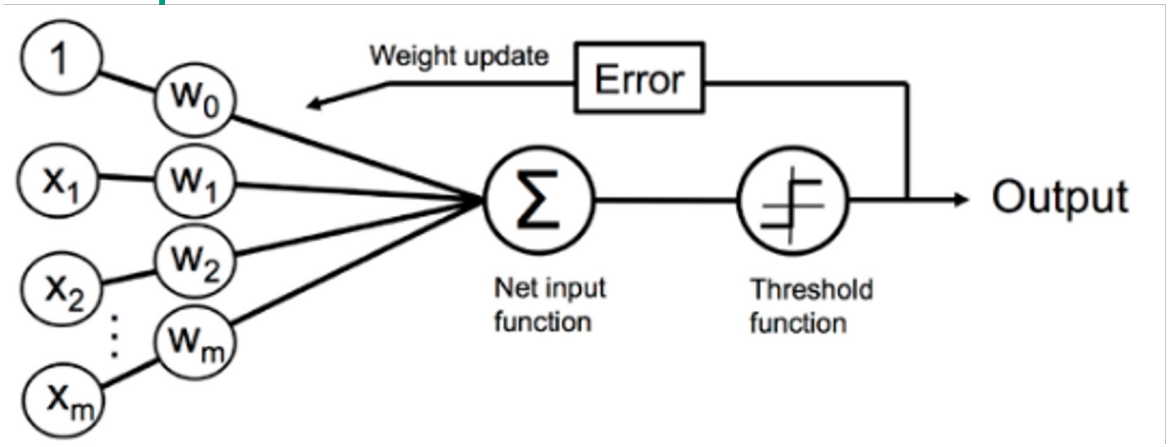


[1]

- The perceptron (first model of an artificial neuron) learning rule is based on the MCP neuron model published in 1957
- The algorithm learns automatically the optimal weight coefficients for the input features
- The product of optimal weights and input features decides whether the neuron fires or not
- Can be used to predict whether an instance belongs to one class or another

[1] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 20

Perceptron concept



- Each perceptron is a set of weights which are multiplied with the set of inputs and summed
- The sum is passed through a threshold function to yield a binary output 1 or -1
- This is then called a *binary classification function*
- The training algorithm computes a decision function based on weights and features
- The goal is to find the optimal weight values for the best possible classification performance

[2] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 26

Formal definition of an artificial neuron

- Binary classification task (two class problem)
  - First class coded as 1 (positive class)
  - Other class: -1 (negative class)
- Decision function (φ(z)):
  - takes linear combinations of
    - values in vector **x**
    - corresponding weights in weight vector **w**
- z: net input

$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

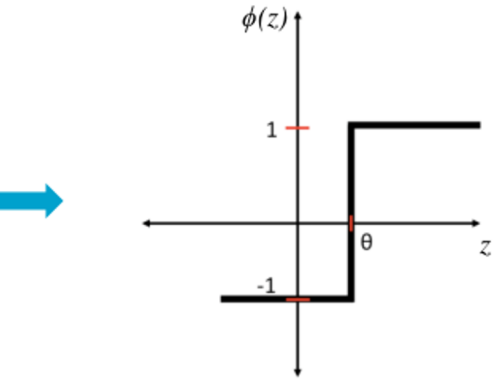
$m \times 1$ $\quad$ $m \times 1$

$$z = w_1 x_1 + \ldots + w_m x_m$$

Formal definition of an artificial neuron

- Given a specific sample $\boldsymbol{x}^{(i)}$
  - If the net input z ≥ θ → predict class 1 for $\boldsymbol{x}^{(i)}$
  - If the net input z < θ → predict class -1 for $\boldsymbol{x}^{(i)}$
  - θ represents the threshold: At which level/value should the sum of all weighted input signals be to predict class 1?
- For the perceptron algorithm φ(·) is a variant of the unit step function

$$\phi(z) = \begin{cases} 1 & if\ z \geq \theta \\ -1 & otherwise \end{cases}$$



Formal definition of an artificial neuron

For simplicity, bring θ to the left side of the equation:

$$z \geq \theta$$

$$w_1 x_1 + w_2 x_2 + \cdots + w_m x_m \geq \theta$$

$$-\theta + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m \geq 0$$

$$-\theta \cdot 1 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m \geq 0$$

$$w_0 x_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m \geq 0$$

where $w_0 = -\theta$

$x_0 = 1$

Formal definition of an artificial neuron

- z can now be written in a more compact form:

$$z = w_0 x_0 + w_1 x_1 + \cdots + w_m x_m = \sum_{j=0}^{m} \boldsymbol{x}_j \boldsymbol{w}_j = \boldsymbol{w}^T \boldsymbol{x}$$

$1 \times (m+1) \quad (m+1) \times 1$

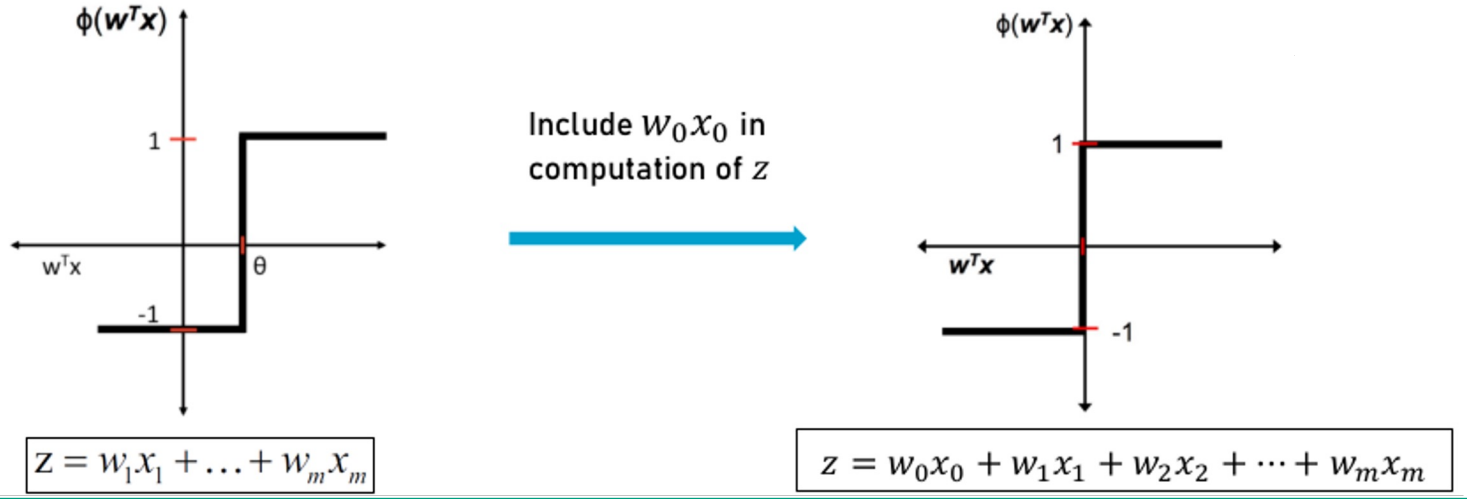
- and $\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$
- In ML literature $w_0 = -\theta$ is often called the **bias unit**
- **Summarised:**
  - **from input values x and weights w → compute the net input z**
  - **from net input z and decision function φ(z) → classify outputs as -1 or 1**

Formal definition of an artificial neuron

- The value of the net input $z = \boldsymbol{w}^T \boldsymbol{x}$ decides whether the decision function of the perceptron produces output 1 or -1.

$$\phi(z) = \begin{cases} 1 \ \ if \ z \geq \theta \\ -1 \ \ otherwise \end{cases} \qquad \phi(z) = \begin{cases} 1 \ \ if \ z \geq 0 \\ -1 \ \ otherwise \end{cases}$$

[3]

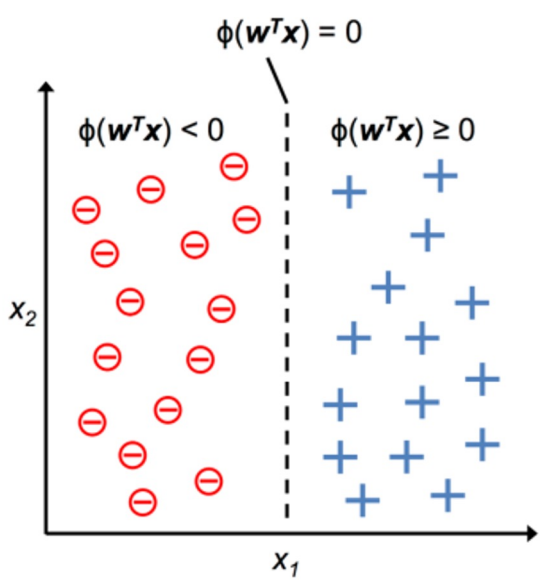


[3] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 23

Formal definition of an artificial neuron

- The value of the net input $z = \boldsymbol{w}^T \boldsymbol{x}$ decides whether the decision function of the perceptron produces output 1 or -1.

[4]



[4] Figure from S. Rascka, V. Mirjalili. *Python Machine learning - 3rd Ed.* (2019), page 23

The Perceptron learning rule

- Idea behind the MCP neuron and Rosenblatt's thresholded perceptron model:
  - Reductionist approach to mimic how a single neuron in the brain works
  - The neuron either fires or not
- Initial perceptron algorithm:
  1. Initialise the weights to 0 or small random numbers
  2. For each training sample $\boldsymbol{x}^{(i)}$
     - 2a. Compute output value $\hat{y}$ (prediction of true class label y)
     - 2b. Compare true class label y and predicted output $\hat{y}$
     - 2c. If different from one another: Update weights
- **How to update the weights?**

The Perceptron learning rule

- The weights w_j in weight vector $\boldsymbol{w}$ are updated simultaneously
- The update of each weight w_j can be more formally written as

$$w_j := w_j + \Delta w_j$$

- The value of Δw_j, which is used to update the weight w_j is calculated by the **perceptron learning rule**

$$\Delta w_j = \eta \left(y^{(i)} - \hat{y}^{(i)} \right) x_j^{(i)}$$

η : Learning rate (typically a constant between 0.0 and 1.0)
$y^{(i)}$: True class label
$\hat{y}^{(i)}$: Predicted class label
$x_j^{(i)}$: Value of feature *j* in sample vector *i*

The Perceptron learning rule

- For a two-dimensional (two variables: x_1 and x_2) dataset:

Prediction/classification error

$$\Delta w_0 = \eta\left(y^{(i)} - output^{(i)}\right)$$

$$\Delta w_1 = \eta\left(y^{(i)} - output^{(i)}\right)x_1^{(i)}$$

$$\Delta w_2 = \eta\left(y^{(i)} - output^{(i)}\right)x_2^{(i)}$$

The Perceptron learning rule

- **Correct** class label predictions:
  - **Negative class** predicted **correctly:** $\Delta w_j = \eta\left(-1-\left(-1\right)\right)x_j^{(i)} = 0$
  - **Positive class** predicted **correctly:** $\Delta w_j = \eta\left(1-1\right)x_j^{(i)} = 0$

- **Incorrect** class label predictions:
  - **Positive class** predicted **incorrectly:** $\Delta w_j = \eta\left(1--1\right)x_j^{(i)} = \eta\left(2\right)x_j^{(i)}$
  - **Negative class** predicted **incorrectly:** $\Delta w_j = \eta\left(-1-1\right)x_j^{(i)} = \eta\left(-2\right)x_j^{(i)}$

The Perceptron learning rule

- Intuition for multiplicative factor $x^{(i)}$ in $\Delta w_j = \eta\left(y^{(i)} - \hat{y}^{(i)}\right)x_j^{(i)}$

- Example: $\hat{y}^{(i)} = -1, \quad y^{(i)} = +1, \quad \eta = 1$
- Assume: $x_j^{(i)} = 0.5$ and sample *i* has been misclassified as -1
- Consequently, the weight update will be: $\Delta w_j = (1 - -1)0.5 = (2)0.5 = 1$
- We see that the weight update *Δw* is proportional to the value of $x^{(i)}$

Important notes on the Perceptron training algorithm

Convergence of the perceptron algorithm is guaranteed only if

The two classes are **linearly** separable

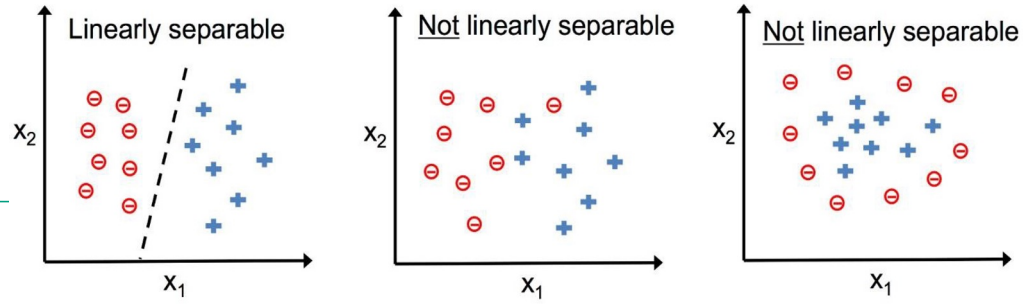
The learning rate is **sufficiently small**

If the classes cannot be separated by a linear decision boundary, take **at least one** of the following two measures:

Set a maximum number of iterations (**epochs**) over the training samples

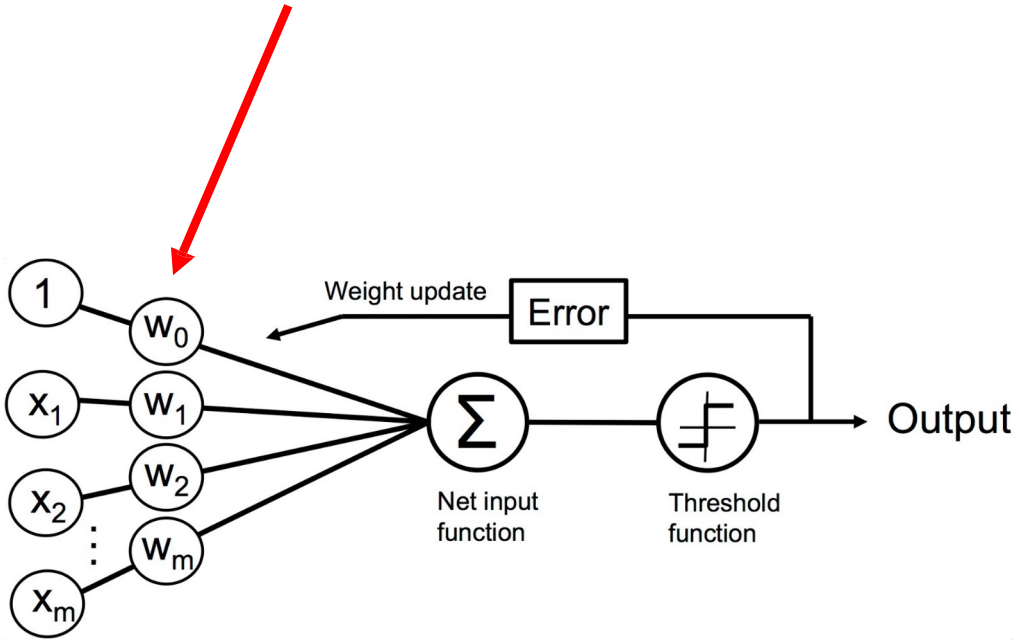
Set a threshold for the maximum number of tolerated misclassifications

If classes are **not** linearly separable **AND** none of the two measures above were taken, the perceptron **never stops** updating weights



Important notes on the Perceptron training algorithm

Goal: Find optimal values for the weights for best possible classification performance



Important notes on the Perceptron training algorithm



- The perceptron class: perceptron.py
- Training perceptron classification on the iris data: Ch02_01_iris_perceptron_alt1.py
- Same as Ch02_01_iris_perceptron_alt1.py, but using the package mlxtend for plotting: Ch02_01_iris_perceptron_alt2.py
- Understanding random seed: Ch_02_play_with_random.py
- Extended version of perceptron.py that collects values of weights across all epochs: perceptron_ext.py
- Extended version of Ch02_01_iris_perceptron_alt1.py visualising weight updates on the iris data: Ch02_03_iris_perceptron_weightPlotting.py

Exercise: Understand the code in perceptron.py



- Open the file: perceptron.py
- Identify where in the code the following is computed
  - Net input
  - Weight updates
  - Threshold function
- Where are the weights initialised?
- What are the dimensions of the prediction?
- How often are the weights updated?
- How often is the net input computed?
- How is the error checked for?

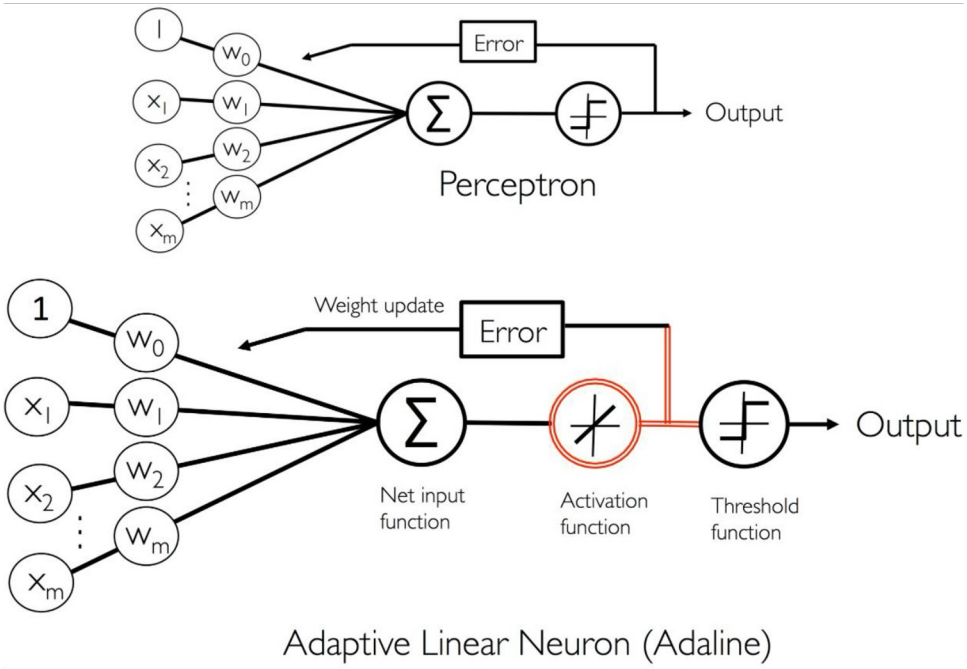
Adaptive Linear Neurons - Learning convergence



- **ADA**ptive **LI**near **NE**uron (Adaline)
- Published in 1960
- Considered to be an improvement over the Perceptron algorithm
- **Key concept** of the Adaline algorithm: **Defining** and **minimising** cost functions
- The concept lays **foundation** for **more advanced machine learning algorithms**
  - Classification models (e.g. logistic regression, Support vector machines (SVM))
  - Regression models
- Introduces the concept of activation functions
  - Various activation functions used in artificial neural networks
  - Logistic, hyperbolic tangent (tanh), rectified linear unit (RELU), etc.

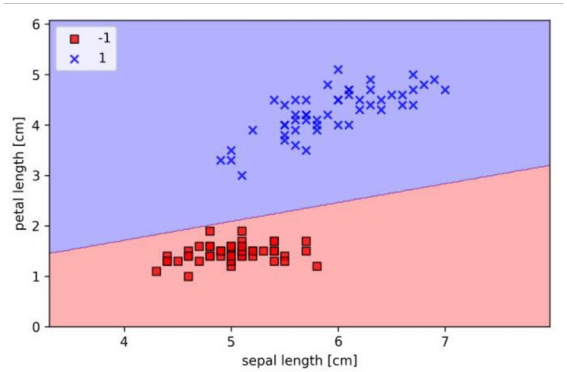
Adaptive Linear Neurons - Learning convergence

- Illustration of the Perceptron and Adaline algorithms:

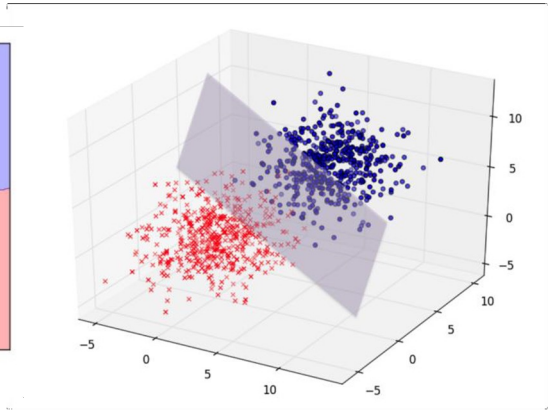


Adaptive Linear Neurons - Learning convergence

- **Common properties** of the Perceptron and Adaline algorithms:
  - Both are classifiers for **binary classification** (two-class problems)
  - Both have a **linear decision boundary**
  - Both use a **threshold function**



Linear decision boundary for two input variables
- a straight line



Linear decision boundary for three input variables
- a plane

More than three input variables

Linear decision boundary for more than three variables
- a hyperplane

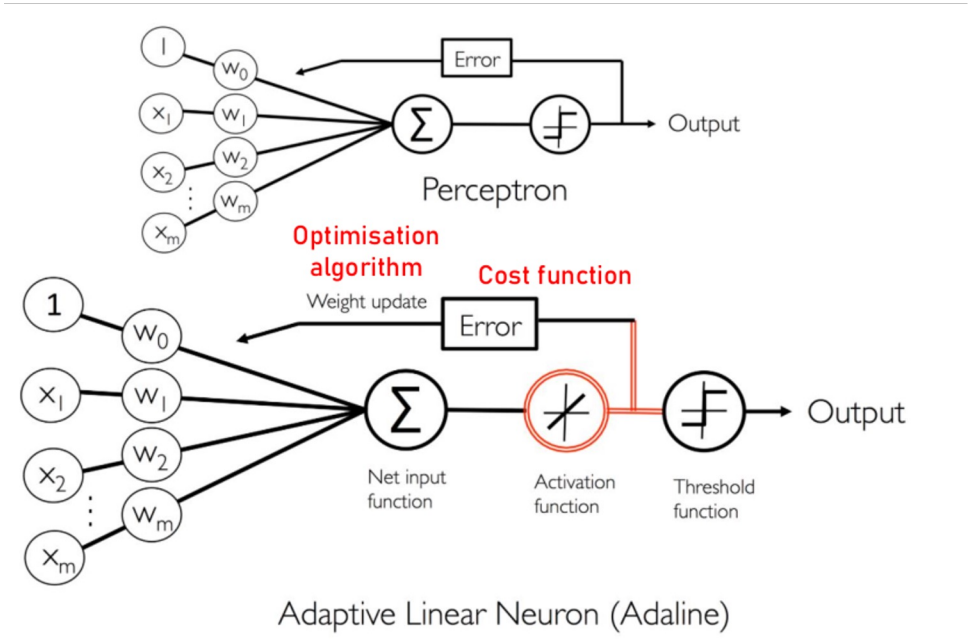
Adaptive Linear Neurons - Learning convergence

- **Key differences** between the Adaline and Perceptron algorithms
- **Perceptron**:
  - Weights are updated using a **step function** $\phi(z)$ (serves as an activation function)
  - Computation of the error: Compares **true class labels** to **predicted class labels**
  - Updates weights **immediately** after a misclassification. Updates happen **multiple times** during a full set of iterations (epoch)
- **Adaline**:
  - Weights are updated based on a **linear activation function** $\phi(z)$
  - The linear activation function is simply an **identity function** of the net input
  - Computation of the error: Compares **true class labels** to the **continuous output** from the activation function
  - Updates weights only **at the end** of each epoch (batch-wise updates)

Linear activation function $\phi(z) = \phi(\boldsymbol{w}^T \boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x}$

Adaptive Linear Neurons - Learning convergence

- A linear activation function is used for learning the weights
- However, a threshold function is still used to make the final prediction
- Illustration of the main differences between the Perceptron and Adaline algorithms:



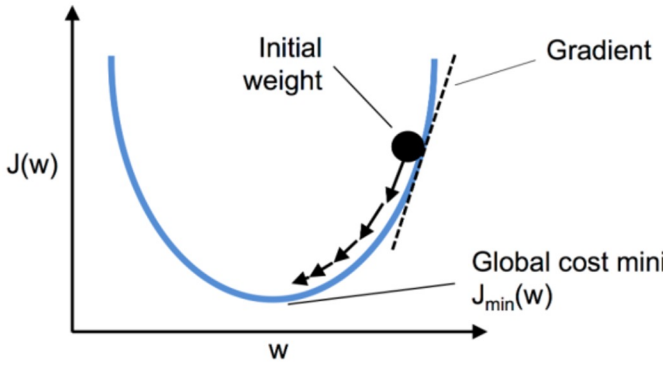
Adaptive Linear Neuron (Adaline)

Minimizing the cost function

- One key ingredient in many machine learning algorithms: A defined objective function (loss function, cost function)
- Goal of using objective/loss/cost function:
  - Capture the performance of the model; how well the model is predicting classes (quality of the output)
  - Doing so by computing the error or distance score between the true class label and the output of the activation function
  - Compute weights such that the loss function is minimised (find the global cost minimum where the error is as small as possible)
- *Learning* means finding a combination of weights that minimizes a loss function for a given set of training samples and their corresponding targets (true class labels)

Minimizing the cost function

- **Adaline algorithm:**
  - Cost function *J* used to learn weights is the Sum of Squared Errors (SSE) between the outputs (from the activation function) and the true class labels



$$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$

Cost function (SSE)

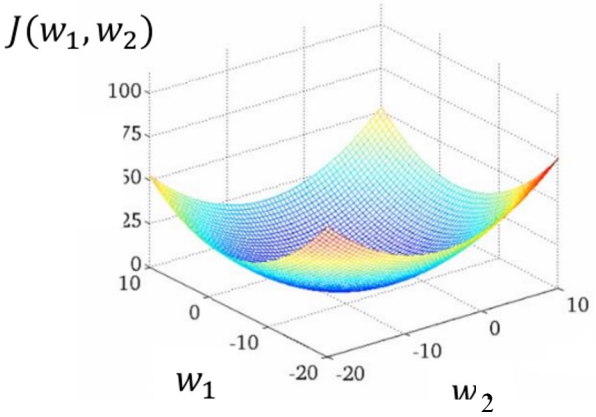
$$\phi(z) = \phi(\boldsymbol{w}^T\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$$

Linear activation function

We can use an *optimisation* algorithm named **Gradient Descent** to find the global minimum of the cost function *J(w)*.

Minimizing the cost function

- **Adaline algorithm:**
  - Cost function *J* used to learn weights is the Sum of Squared Errors (SSE) between the outputs (from the activation function) and the true class labels



$$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$

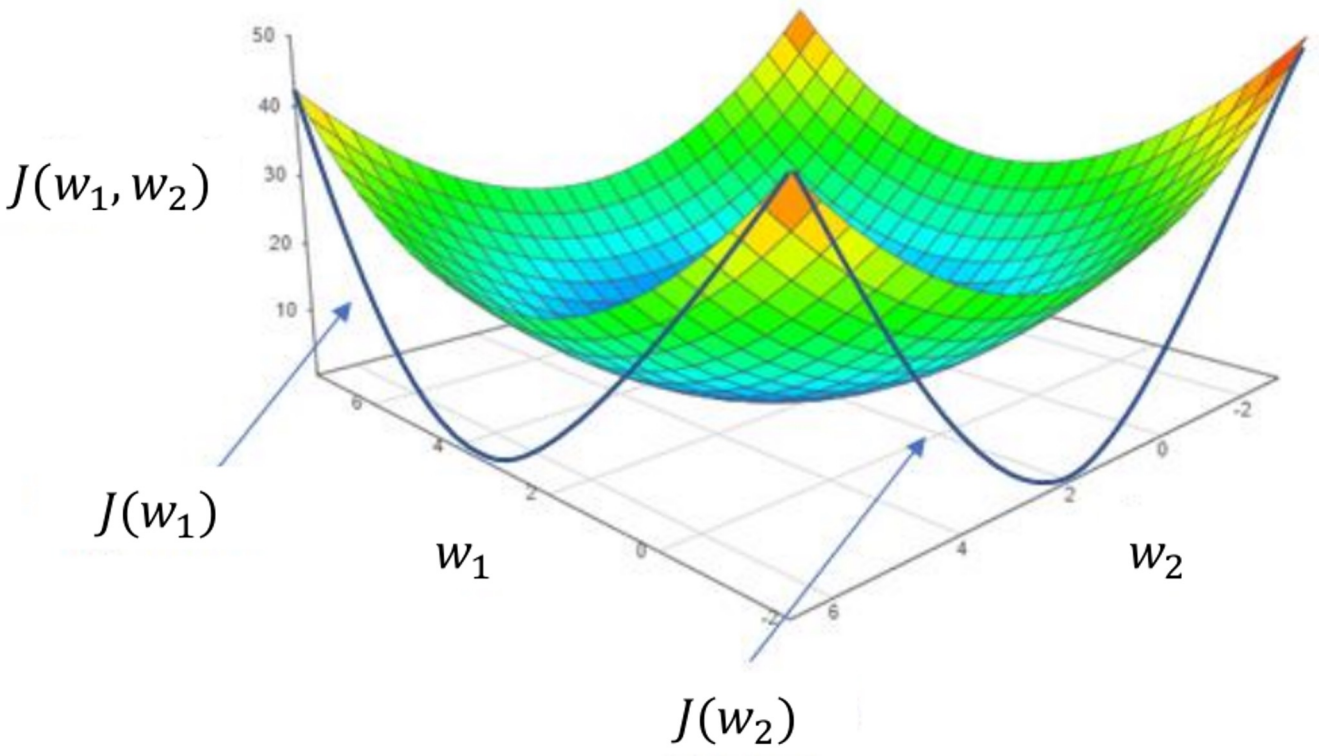
Cost function (SSE)

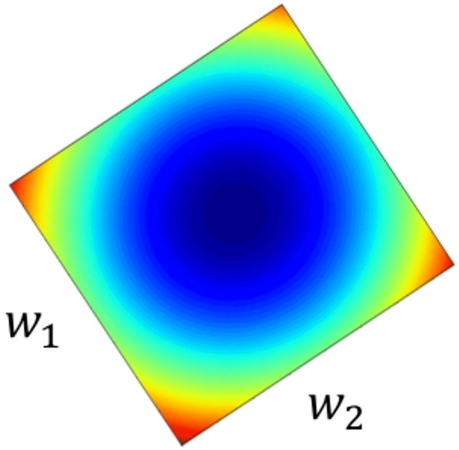
$$\phi(z) = \phi(\boldsymbol{w}^T\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$$

Linear activation function

We can use an *optimisation* algorithm named **Gradient Descent** to find the global minimum of the cost function *J(w)*.

Minimizing the cost function





Minimizing the cost function

- The main advantage of the linear activation function over the unit step function is that the cost function becomes **differentiable**

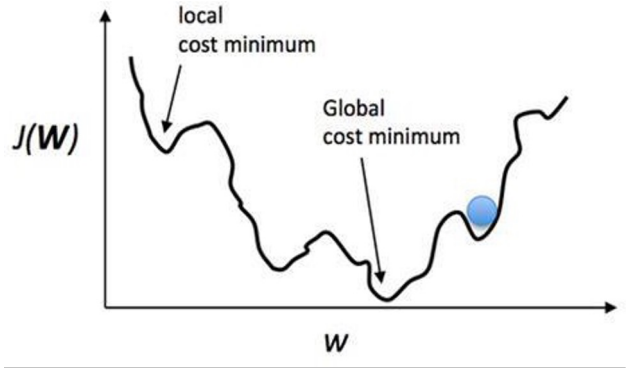
  -> Allows us to derive the gradient of the loss function (important information for moving in the right direction towards the global cost minimum)

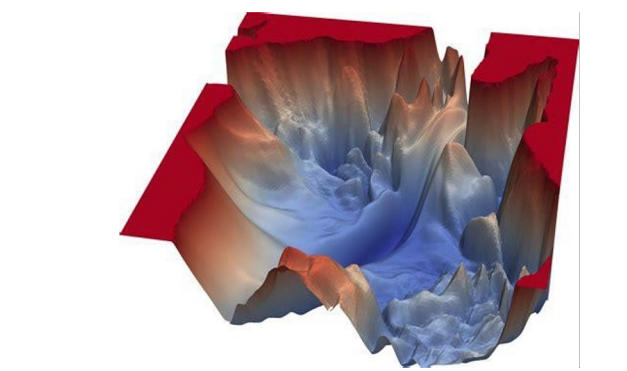
- A convenient property of the SSE cost function: It is convex

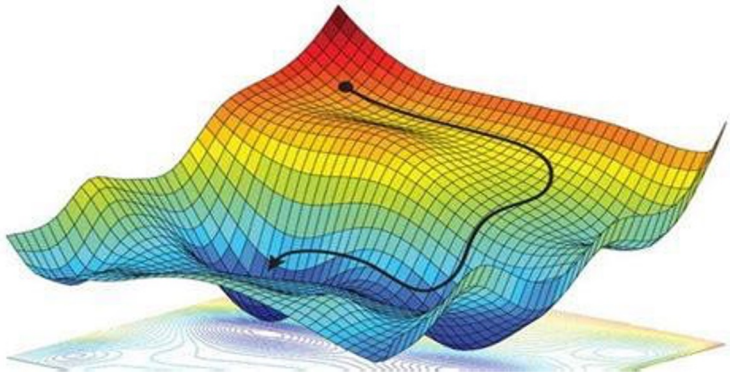
  -> Enables the gradient descent algorithm to move towards the weights that result in a global cost minimum

Minimizing the cost function

Examples of loss landscapes (not Adaline):







Minimizing the cost function

- Adaline algorithm:
  - Cost function *J* learns weights from the Sum of Squared Errors (SSE) between the calculated outcomes and the true class labels: $$J(\boldsymbol{w}) = \frac{1}{2}\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right)^2$$
- Using gradient descent updates the weights by taking a step in the opposite direction of the gradient of the cost function $\nabla J(w)$: $$\boldsymbol{w} := \boldsymbol{w} + \Delta \boldsymbol{w}$$ $$\Delta \boldsymbol{w} = -\eta \nabla \mathrm{J}(\boldsymbol{w})$$
- The weight change Δw is defined as the negative gradient $\nabla J(w)$ multiplied by the learning rate η
- Update of the weights: $$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right) x_j^{(i)}$$ $$\frac{\partial J}{\partial w_j} = -\sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right) x_j^{(i)}$$

Minimizing the cost function



The squared error derivative

If you are familiar with calculus, the partial derivative of the SSE cost function with respect to the *j*th weight can be obtained as follows:

$$\frac{\partial J}{\partial w_j} = \frac{\partial}{\partial w_j}\frac{1}{2}\sum_i \left(y^{(i)} - \phi(z^{(i)})\right)^2$$

$$= \frac{1}{2}\frac{\partial}{\partial w_j}\sum_i \left(y^{(i)} - \phi(z^{(i)})\right)^2$$

$$= \frac{1}{2}\sum_i 2\left(y^{(i)} - \phi(z^{(i)})\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \phi(z^{(i)})\right)$$

$$= \sum_i \left(y^{(i)} - \phi(z^{(i)})\right)\frac{\partial}{\partial w_j}\left(y^{(i)} - \sum_i \left(w_j^{(i)} x_j^{(i)}\right)\right)$$

$$= \sum_i \left(y^{(i)} - \phi(z^{(i)})\right)\left(-x_j^{(i)}\right)$$

$$= -\sum_i \left(y^{(i)} - \phi(z^{(i)})\right)x_j^{(i)}$$

Chain Rule

$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$

Minimizing the cost function

- Adaline algorithm:
  - Update of the weights:

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)} \right) \right) x_j^{(i)}$$

- A single weight update is computed based on all samples in the training set (instead of incrementally after each sample as with the Perceptron algorithm)

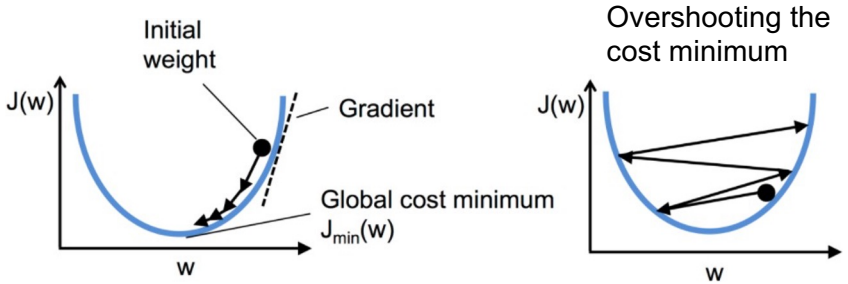
  -> Referred to as **batch gradient descent**

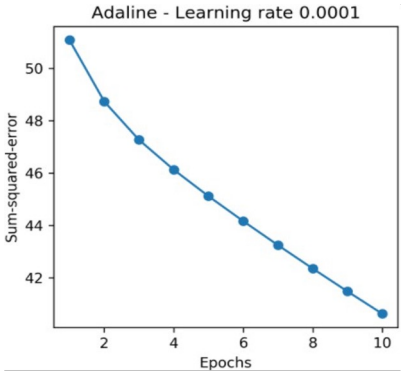
- Note the similarity with the weight updates of the Perceptron algorithm:
  - Perceptron: $\phi(z)$ takes values -1 or 1
  - Adaline: $\phi(z)$ takes continuous values $\boldsymbol{w}^T \boldsymbol{x}$

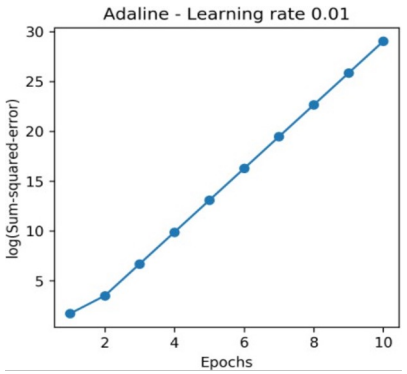
Minimizing the cost function

Error over a number of epochs –> important to find the appropriate learning rate η

for the weight updates Δw







$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i \left(y^{(i)} - \phi\left(z^{(i)}\right)\right) x_j^{(i)}$$

`adaline.py`
`Ch02_04_iris_adaline.py`

Improving Gradient Descent with Feature Scaling

- Many machine learning algorithms require some sort of feature scaling for optimal performance (also called standardisation)
- Gradient descent is one of the many algorithms that benefit from feature scaling

- Scaling (standardisation) gives the data some specific properties
  - Standard distribution -> helps the gradient descent learning to converge more quickly
  - Shifts the mean of each feature so that it is centered at zero
  - Each feature has a standard deviation of 1

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

x_j: feature vector (variable or column in data)
μ_j: mean of feature vector
σ_j: standard deviation of feature vector
x'_j: scaled feature vector

Improving Gradient Descent with Feature Scaling

Person	Height (cm)	Weight (kg)	Shoe size
Person A	174	55	46
Person B	188	92	45
Person C	158	65	42
Person D	202	110	49
Person E	171	96	44
Person F	193	79	48
Mean	181	82.833333	45.6667
STD	16.198765	20.507722	2.58199

Original data

Person	Height (cm)	Weight (kg)	Shoe size
Person A	-7	-27.833333	0.33333
Person B	7	9.1666667	-0.66667
Person C	-23	-17.833333	-3.66667
Person D	21	27.166667	3.33333
Person E	-10	13.166667	-1.66667
Person F	12	-3.8333333	2.33333
Mean	0.00	0.00	0.00
STD	16.198765	20.507722	2.58199

Centred data (zero mean)

Person	Height (cm)	Weight (kg)	Shoe size
Person A	-0.4321317	-1.3572123	0.1291
Person B	0.4321317	0.4469861	-0.2582
Person C	-1.4198613	-0.8695911	-1.42009
Person D	1.2963951	1.3247043	1.29099
Person E	-0.617331	0.6420346	-0.6455
Person F	0.7407972	-0.1869215	0.9037
Mean	0.00	0.00	0.00
STD	1	1	1

Standardised (scaled) data

Improving Gradient Descent with Feature Scaling

- Number of objects (rows):
  - *i = 1 … N*
- Number of variables (columns):
  - *j = 1 … K*
- Observed value x_{ij} for
  - i'th sample
  - j'th variable

$$X = \begin{pmatrix} x_{11} & \cdots & x_{1K} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{NK} \end{pmatrix}$$

center

$$x_{ij,cent} = x_{ij} - \bar{x}_j$$

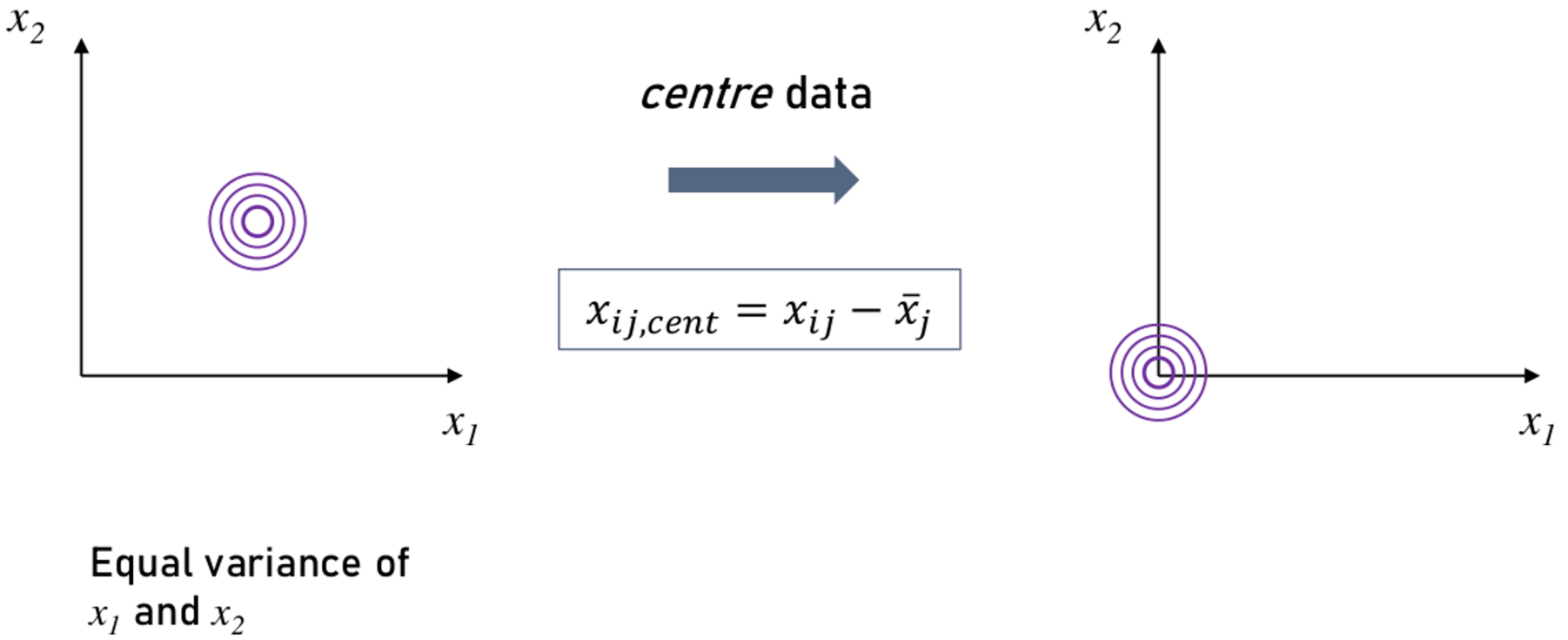
$$\bar{x}_j = \frac{1}{N}\sum_{j=1}^{N} x_{ij}$$

standardise

$$x_{ij,stand} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$$

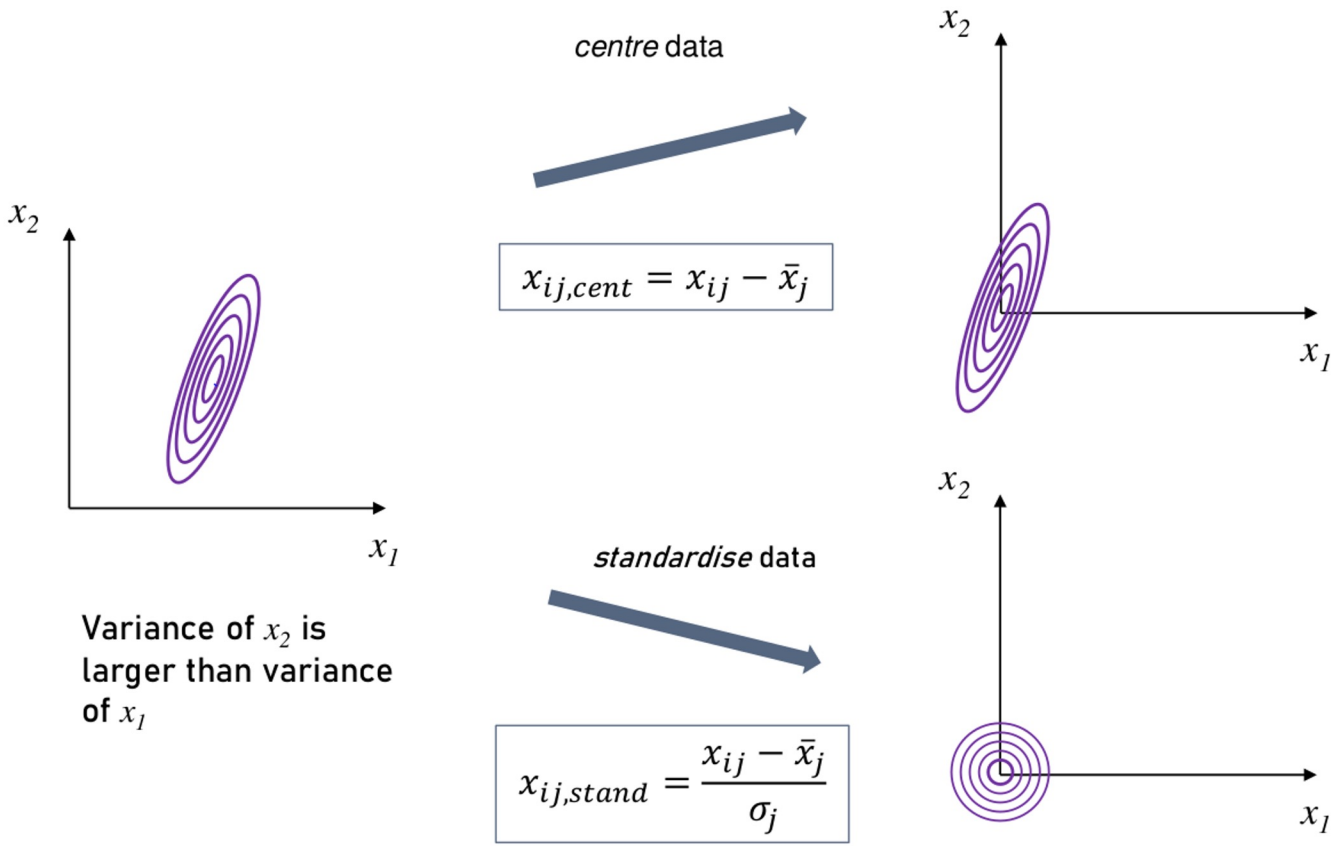
$$\sigma_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(x_{ij} - \bar{x}_j\right)^2}$$

Improving Gradient Descent with Feature Scaling



Equal variance of
x_1 and x_2

Improving Gradient Descent with Feature Scaling



Improving Gradient Descent with Feature Scaling



- Feature scaling (standardisation) helps the gradient descent (our optimiser) to find a good or optimal solution in fewer steps
- Note that SSE (cost function) may remain non-zero even though all samples were classified correctly

Improving Gradient Descent with Feature Scaling

- Feature scaling with numpy – one feature at a time

```
 99 # Standardise X one feature at a time (not very efficient)
100 X_sc = X.copy()
101 X_sc[:, 0] = (X[:, 0] - X[:, 0].mean()) / X[:, 0].std()
102 X_sc[:, 1] = (X[:, 1] - X[:, 1].mean()) / X[:, 1].std()
```

- Feature scaling with numpy – all variables at once

```
105 # Standardise X (all features at once)
106 X_sc = X.copy()
107 X_sc = (X_sc - np.mean(X_sc, axis=0)) / np.std(X_sc, axis=0)
```

```
Ch02_05_iris_perceptron_weightPlotting_selectClasses_scaling.py
Ch02_06_iris_adaline_weightPlotting_selectClasses_scaling.py
Ch02_07_scaling_examples.py
```

Thank you for coming!

